

- **Multiple storage engines:**

It supports multiple storage engines in order to save the data in memory as well as disk.

The R programming language contains several interfaces in order to connect with MongoDB. With the R interfaces, we are able to extract data and perform data analytics tasks.

R packages for MongoDB

R provides several packages to deal with MongoDB, which are listed below.

- **mongolite:** This is a fast and very simple MongoDB client for R programming. It is based on the *mongo-c-driver* and *jsonlite*. It has several functionalities like indexing, encryption, Map-Reduce, aggregation, streaming, etc.
- **RMongo:** This is a user friendly MongoDB database interface for R. The interface is provided via Java calls to the *mongo-java-driver*.
- **rmongodb:** This is a very powerful package in order to interact with MongoDB. It provides functionalities like querying, inserting and updating to MongoDB with JSON and BSON, handling BSON objects, creating indices on MongoDB collections, aggregation pipelines, etc.

The mongolite package for MongoDB

Mongolite can be installed directly from CRAN for the Windows operating system, as follows:

```
install.packages("mongolite")
```

You need to set up a local server for MongoDB. Once the mongolite package is installed, load it by using

the following command:

```
library(mongolite)
```

The *mongo()* function connects to a MongoDB server. The parameters for the function are:

```
mongo(dataset, url)
```

For example:

```
Test <- mongo("mydataset", url=
"mongodb://cmpica:admin@mango.cmpica.
org:40123/mongo_test")
```

Once it is connected, initiate it by establishing a new connection:

```
Newc <- mongo("Result")
Newc $ insert (ggplot2:: Result)
```

Once data insertion is over, we can fire queries on the data set. JSON based syntax is used to query the data set.

```
Newc$count('{}')
// {} means all data selection
> [1] 270
Readdata <- Newc $ find('{}')
// Read all the data
Print(Readdata)
// Display the data
Q1 <- Newc$ find ( ' { "Semester" : "
First", "CGPA" : { "$gt" : 8 } } )

// Retrieve all records having First
```

Semester and CGPA > 8

The *find()* method is applicable for the collection of records. Another function *iterate()* allows you to perform a query — this is not for the collection of records, but it reads the records one-by-one without any collection. The iterator has methods *one()*, *batch(n)* which allow you to step through a single or *n* number of records at a time.

```
Q2 <- Newc$ iterate ( ' { "Semester" :
" First", sort = '{ "CGPA" : 1 }' )
while(! is.null ( u1 <- 10) ) {
cat (sprintf( " Result of First
Semester is %d CGPA", u1 $CGPA)
}
```

```
// Read the records from the iterator
```

R programming contains a number of packages that deal with MongoDB. In this article we have covered three main packages and their utilities. Mongolite, a fast and very simple MongoDB client from all three packages has been discussed in detail. We have explored how to install and load the packages, and how to initiate the connection with MongoDB, establish the new connection with the *mongo* function, insert the data using the *insert* method, and how to fire the queries. **END** 🐧

References

- [1] <https://jeroen.github.io/mongolite/>
- [2] <https://datascienceplus.com/using-mongodb-with-r/>
- [3] <https://rdrr.io/cran/mongolite/man/mongo.html>
- [4] <https://www.kenwalger.com/blog/nosql/mongodb/using-r-mongodb/>
- [5] <https://www.mongodb.com/presentations/using-r-for-advanced-analytics-with-mongodb>

By: Dr Dharmendra Patel and Dr Atul Patel

The authors are associated with the Smt. Chandaben Mohanbhai Patel Institute of Computer Applications, Charusat, Gujarat. Their areas of interest are data mining, data science, artificial intelligence, deep learning and image processing.